the system. Ideally, the performance increase is linear, so a system with 100 processors performs 100 times better than a system with one processor.

Although code can work even if it isn't scalable—a single-threaded application is certainly not scalable, for example—exception safety is a matter of correctness. If your code isn't exception safe, you can end up with broken invariants or race conditions, or your application might terminate unexpectedly because an operation threw an exception. With this in mind, we'll look at exception safety first.

8.4.1. Exception safety in parallel algorithms

Exception safety is an essential aspect of good C++ code, and code that uses concurrency is no exception. In fact, parallel algorithms often require that you take more care with regard to exceptions than normal sequential algorithms. If an operation in a sequential algorithm throws an exception, the algorithm only has to worry about ensuring that it tidies up after itself to avoid resource leaks and broken invariants; it can merrily allow the exception to propagate to the caller for them to handle. By contrast, in a parallel algorithm many of the operations will be running on separate threads. In this case, the exception can't be allowed to propagate because it's on the wrong call stack. If a function spawned on a new thread exits with an exception, the application is terminated.

As a concrete example, let's revisit the `parallel_accumulate` function from listing 2.8, which is reproduced here.

Listing 8.2. A naïve parallel version of `std::accumulate` (from listing 2.8)